



Software Engineering Department
Braude College of Engineering, Karmiel
Capstone Project – Phase B (61999)

Sen2Pro

A Probabilistic Perspective on Sentence Embedding for Authorship Verification

Team 26-1-R-21

Nikita Konovalenko 321237406
Kfir Cohen 316112176

Advisors

Dr. Renata Avros & Prof. Zeev Volkovich

Project Repository (Git – Phase A & B books, full code, poster, video):

github.com/KfirCohen-PyLab/Final_Degree_Project_Phase_A

[replace with the Phase B repository link if it differs]

Table of Contents

Abstract

Sen2Pro is an authorship-verification system that combines probabilistic sentence embeddings with the Deep-Impostors stylometric pipeline to investigate the Shakespeare authorship question. Instead of representing each text segment as a single fixed vector, the system models it as a Gaussian distribution $N(\mu, \text{diag}(\sigma^2))$, where the mean captures the central stylistic tendency and the per-dimension variance captures the uncertainty of the representation. Uncertainty is estimated at the encoder level via Monte-Carlo Dropout and, optionally, through text perturbations that emulate data uncertainty.

In Phase B we implemented the full pipeline end-to-end: probabilistic embedding generation with `all-distilroberta-v1`, a CNN+BiLSTM impostor separator trained per author pair, sliding-window stylistic-signal smoothing, normalized Dynamic Time Warping (DTW) over within-document signals, K-medoids clustering, and an Isolation-Forest anomaly readout. We evaluated every configuration under a leave-one-document-out (LOO) protocol on 50 Shakespeare texts and 30 impostor authors. The best operational model reaches **AUC = 0.989** and **0.903 balanced accuracy (LOO)**, with Shakespeare recall 0.94 and impostor rejection 0.87 at a calibrated threshold. We additionally wrapped the decision in a class-conditional conformal predictor that delivers calibrated coverage with explicit *abstain* and *outlier* options — a natural fit for genuinely disputed texts. A central, honestly reported finding is that the deterministic mean representation is already a very strong baseline; encoder-level uncertainty carries limited but non-zero additional authorship signal on this corpus.

1. Problem Overview and Background

Authorship verification asks whether a questioned document was written by a particular author, based on stylistic and linguistic patterns rather than content. The problem is central to forensic linguistics, literary scholarship, and digital security, where it underpins plagiarism detection, impersonation detection, and the resolution of historical authorship disputes. Its core difficulty is that writing style is a nuanced combination of many interacting features that are shaped by genre, historical period, collaboration, and deliberate imitation, and that drift within a single author over time.

The **Shakespeare authorship question** — the long-standing debate over the extent to which the works of the Shakespeare canon were written by a single author — is a canonical benchmark for computational stylometry. It pairs a rich, well-studied corpus with a genuinely open question, making it an ideal stress test for methods that aim to surface latent stylistic structure not easily seen by manual inspection.

This project builds directly on two methodological strands reviewed in Phase A. The first is **Sen2Pro** (Shen et al., 2023), which represents a sentence as a Gaussian distribution rather than a point, capturing both semantic meaning and representational uncertainty. The second is the **Deep-Impostors** framework (Volkovich & Avros, 2025), which trains separators between pairs of “impostor” authors, scores a questioned text against them, and uses anomaly detection over the resulting stylistic signals to decide authenticity. The objective of Phase B was to integrate these two ideas into a single working system and to measure honestly whether the probabilistic representation improves verification.

This chapter condenses the introduction and literature survey of the Phase A book. The full survey, including the mathematical derivations of the Gaussian embedding and the Deep-Impostors algorithm, appears in the Phase A document available in the project repository.

2. What We Built: General Description

2.1 System goal and users

Sen2Pro is a research-oriented software framework that takes one or more raw text documents and returns, for each, a **calibrated Shakespeare-affinity score** together with a verdict (authentic / suspicious / impostor), an uncertainty estimate, and — in conformal mode — a coverage-guaranteed decision set. The intended users are **researchers and students in computational stylometry and digital humanities**: people who want to run reproducible authorship experiments, compare deterministic and probabilistic representations, and interrogate disputed texts rather than consumers seeking a one-click verdict. Accordingly the system is delivered as a documented, parameterised Google Colab / Jupyter notebook with a single configuration cell as the source of truth, plus a clean prediction API for scoring new texts.

2.2 Character of the system

The system is a **batch analytical pipeline**, not an interactive service. It is organised as a linear sequence of reproducible steps (load → segment → embed → train separators → generate signals → cluster / verify → report), each of which persists its artefacts so that later steps can be re-run without recomputing expensive embeddings. Determinism is enforced through a single global random seed, and every expensive knob (number of Monte-Carlo passes, number of impostor pairs, perturbation strength) is exposed in one hyperparameter cell.

2.3 Data

The corpus consists of **50 Shakespeare texts** and **30 impostor authors** (110 texts) drawn mostly from Project Gutenberg, spanning contemporaries (Marlowe, Jonson, Kyd, Bacon, Donne, Milton), later canonical authors (Austen, the Brontës, Dickens, Hardy, Eliot, Woolf), and modern writers (Christie, Asimov, Clarke, Sheckley), plus a few control sets. After Gutenberg boilerplate removal and segmentation into 50-word units, the corpus yields **22,703 Shakespeare segments** and **182,476 impostor segments**.

3. The Solution: Architecture, Algorithms and Method

Because this is a **research-track** project, this section describes both the architecture of the software we built and the algorithmic method it implements. The high-level design follows the Sen2Pro pipeline proposed in Phase A; the figure below reproduces that conceptual architecture, after which we describe the concrete components actually implemented and the points where the implementation deliberately diverges from the original proposal.

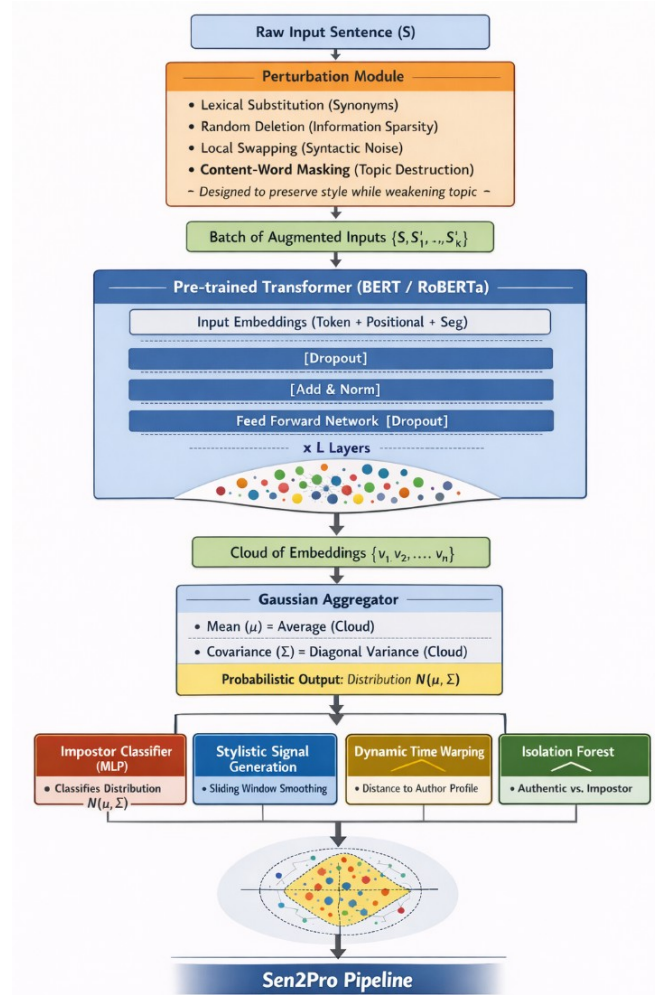


Figure 1. The proposed Sen2Pro pipeline (Phase A design): perturbation → probabilistic transformer embeddings → Gaussian aggregation → impostor separation, stylistic-signal generation, DTW and Isolation Forest.

3.1 Probabilistic sentence embeddings

Each 50-word segment is encoded into a Gaussian $N(\mu, \text{diag}(\sigma^2))$ rather than a single vector. We use the all-distilroberta-v1 encoder (a RoBERTa-family sentence transformer, 768-dimensional output) and adopt **first-last layer average pooling**, in line with the stability analysis in Phase A (Section 2.2.1). Two stochastic sources feed the Gaussian estimate:

- **Model uncertainty:** the encoder is run $\text{EMB_MC_ITERATIONS} = 25$ times with dropout active at inference (Monte-Carlo Dropout; Gal & Ghahramani, 2016). The spread of the resulting embeddings quantifies the model's own representational uncertainty.
- **Data uncertainty:** optionally, each segment is perturbed $\text{N_PERTURBATIONS} = 3$ times (lexical substitution, random deletion, local swapping, and content-word masking); the original and perturbed variants are embedded together and pooled into the same Gaussian, emulating the natural variability of paraphrase.

The mean and a **diagonal** covariance are estimated by the method of moments: $\mu = (1/N) \sum x_i$ and $\sigma^2 = (1/N) \sum (x_i - \mu)^2$. The diagonal approximation trades cross-dimensional correlations for numerical stability

and tractable distance computation, exactly as motivated in Phase A. A small floor $\text{COV_EPS} = 1e-6$ (plus author-specific smoothing λ) keeps every covariance strictly positive-definite.

3.2 Encoder backbone

The encoder is a pre-trained Transformer used purely as a feature extractor. The figure below recalls the canonical Transformer/BERT architecture that underlies the RoBERTa encoder. Self-attention provides global context without recurrence, and first-last layer averaging yields a stable sentence representation on which the probabilistic estimation is built.

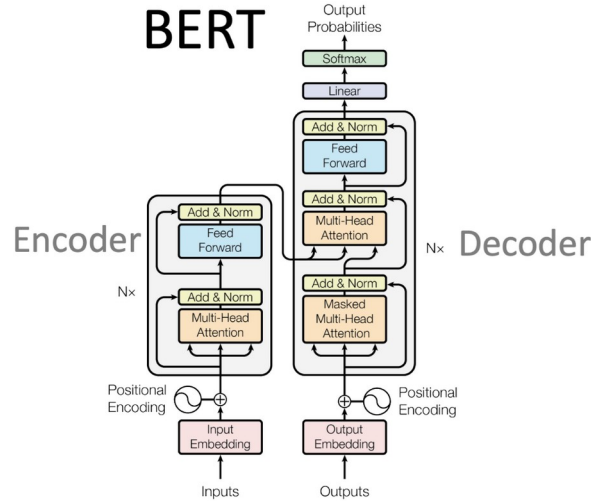


Figure 2. Transformer / BERT encoder architecture (Vaswani et al., 2017). The implementation uses an encoder-only RoBERTa variant, all-distilroberta-v1.

3.3 The impostor separator (CNN + BiLSTM)

For every sampled pair of impostor authors (Z_i, Z_j) we train a binary separator that learns to tell their styles apart from windows of probabilistic features. **Here the implementation diverges from the Phase A proposal in an important, deliberate way:** the Phase A figure named a simple MLP impostor classifier, but we implemented the stronger **CNN + BiLSTM** separator from the Deep-Impostors reference (Volkovich & Avros, 2025):

Conv1D(k=3,6,12) → MaxPool → AdaptiveAvgPool(4) → BiLSTM(500) → BiLSTM(250) →
Dense(2048) → Dense(1)

The separator input is the per-segment probabilistic feature vector. In `mean_only` mode this is μ (768-d); in `mean_logvar` mode it is $\text{concat}(\mu, w \cdot \log(\sigma^2))$ (1536-d), so the variance information enters the model explicitly as extra feature dimensions weighted by `LOGVAR_WEIGHT`. The separator is trained for 5 epochs and only kept if its training accuracy clears the 60% quality gate from the reference paper. Crucially, the CNN is used **deterministically** at inference — all uncertainty in the method lives at the encoder level, and CNN-level MC Dropout is intentionally disabled.

3.4 Stylistic signals with sliding windows

Applying a trained separator to a questioned text's windows yields a sequence of per-window probabilities, which we turn into a continuous **stylistic signal**. Rather than averaging over disjoint chunks (which injects boundary artefacts), we use overlapping **sliding windows** of width `CHUNK_SIZE` =

8 with stride 1: $S(k) = (1/w) \sum p_t$, for $t = k \dots k+w-1$. This convolves the discrete probabilities with a uniform kernel, attenuating high-frequency noise, preserving local stylistic continuity, and diluting outlier segments — producing a smoother, longer signal that is far better suited to alignment-based comparison.

3.5 Document comparison: normalized DTW and clustering

Documents are compared by aligning their stylistic signals with **Dynamic Time Warping (DTW)**. We z-score each signal before alignment (so DTW compares *shape*, not mean/scale offsets) and divide the alignment cost by the path length (removing length bias) — “normalized DTW over stylistic signals.” DTW handles local variation in stylistic pacing that a rigid point-to-point distance would miss. The figure below recalls how DTW non-linearly warps two signals onto a common timeline.

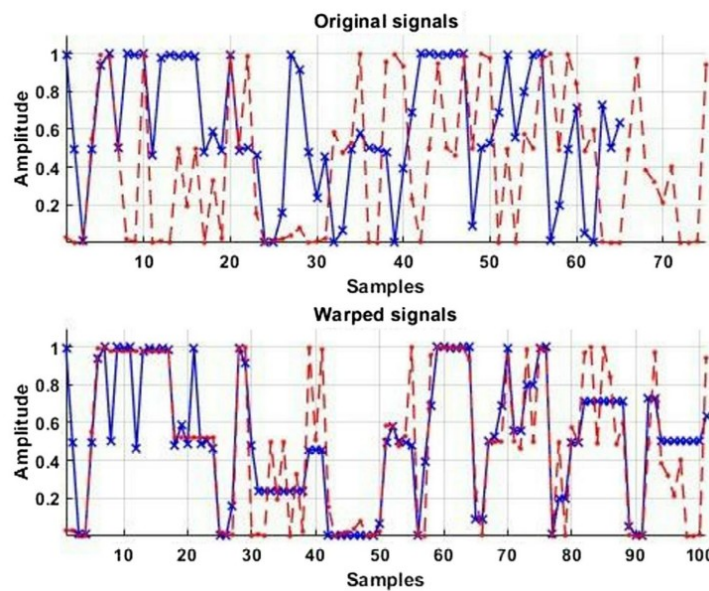


Figure 3. Dynamic Time Warping: original signals (top) and their warped alignment (bottom). DTW aligns stylistic curves non-linearly before computing a distance (Volkovich & Avros, 2025).

The averaged DTW distances over impostor pairs feed a **K-medoids** clustering that separates documents into an authentic and a suspicious cluster, snapshotted every few impostor pairs to show the decision stabilising. In parallel, an **Isolation Forest** produces a per-document anomaly score as a separate analytical readout. With the default `SIGNAL_DISTANCE = dtw_window`, the **final clustering is decided by K-medoids on the normalized-DTW distances**; the Isolation-Forest score is reported as a companion signal, not the decider.

3.6 Calibrated and conformal verification

On top of the clustering pipeline we built a leakage-safe, data-driven **verifier** that fits diagonal Gaussians on random subspaces of the probabilistic features and ranks a questioned document against the impostor pool. Because the raw score is a rank statistic, a fixed 0.5 cut is rarely optimal, so we calibrate the decision threshold on a grid using balanced accuracy, and — to avoid optimism — report a **leave-one-document-out nested-calibration** number as the defensible headline. Finally, a **class-conditional (Mondrian) conformal predictor** converts the score into a decision set with a coverage guarantee: at level α the output is a confident label, an *abstain* (both labels plausible), or an

outlier (matches neither — a natural “novelty / disputed-text” flag). This is well suited to the Shakespeare question, where some texts may genuinely be out-of-distribution.

4. Development Process

Phase B translated the Phase A design into a fully operational framework. We worked in incremental, independently runnable stages, validating each before moving on, and kept every configuration knob in a single source-of-truth cell so that experiments were reproducible by changing one value and re-running from the affected step.

4.1 Work stages

1. **Research & theoretical design (Phase A – completed):** literature survey, the Sen2Pro–Deep-Impostors mathematical formulation, and the architecture documented in the Phase A book.
2. **Implementation & integration:** coding the probabilistic embedding generator and integrating it with the CNN+BiLSTM separator, the sliding-window signal generator, normalized DTW, K-medoids and Isolation Forest; wiring the single hyperparameter cell and the artefact-saving layer.
3. **Experimental validation:** running the LOO evaluation suite — log-variance weight sweep, threshold calibration, normalization and distance comparisons, perturbation sweep, λ sensitivity, pooling validation, and the encoder-level uncertainty ablation.
4. **Analysis & optimization:** automatic best-model selection by honest LOO balanced accuracy, conformal calibration, sanity checks, and the writing of this report.

4.2 Tools and environment

The system is implemented in **Python** and runs on **Google Colab Pro** with a CUDA GPU (the production runs used an NVIDIA RTX PRO 6000 Blackwell, 102 GB VRAM; PyTorch 2.11, CUDA 12.8). Key libraries: sentence-transformers / HuggingFace (encoder + MC Dropout), PyTorch (CNN+BiLSTM separator, mixed-precision bfloat16), scikit-learn (Isolation Forest, PCA), fastdtw (DTW), nltk (tokenisation), and numpy / scipy / matplotlib / seaborn. The dataset lives on Google Drive; artefacts are pickled back to Drive after each run.

4.3 Interaction with the client (advisors)

Development was guided by the project advisors, Dr. Renata Avros and Prof. Zeev Volkovich, whose Deep-Impostors work is the methodological basis of the project. Progress was reviewed iteratively against the reference method; advisor feedback drove several concrete decisions, including the adoption of the CNN+BiLSTM separator over the lighter classifier sketched in Phase A, the choice of normalized DTW over within-document signals as the main document distance, and the emphasis on an honest LOO protocol rather than optimistic in-sample numbers.

4.4 Generative-AI tools used

In line with academic-integrity guidelines, AI tools were used strictly for auxiliary support — literature summarisation, Python boilerplate and debugging, and prose refinement — with all algorithmic logic and final verification performed by the team: **Google Gemini** (paper summarisation, architecture brainstorming), **ChatGPT** (boilerplate and debugging), **Claude** (academic-tone editing), and

DeepSeek (deep-learning / optimisation queries). A detailed prompt log is included in the Phase A book.

5. Requirements and How They Were Met

The functional and non-functional requirements specified in Phase A were implemented as follows. Each requirement maps onto a concrete, runnable stage of the pipeline.

#	Requirement (Phase A)	Status in Phase B
FR1	Accept raw text and preprocess	Done — robust multi-encoding loader + Gutenberg boilerplate stripping
FR2	Segment into consecutive units	Done — non-overlapping 50-word segments ($L = 50$)
FR3	Generate probabilistic embeddings	Done — $N(\mu, \text{diag}(\sigma^2))$ via encoder MC Dropout (+ optional perturbations)
FR4	Segment-level authorship classification	Done — CNN+BiLSTM separator per impostor pair
FR5	Temporal smoothing	Done — sliding-window stylistic signal $S(k)$
FR6	Anomaly detection over the signal	Done — normalized DTW + K-medoids (decider) + Isolation Forest (readout)
NFR ₁	Process a 2000-word doc within ~1 min	Met on the target GPU; segmentation + cached embeddings keep per-doc scoring fast
NFR ₂	Graceful handling of invalid input	Met — encoding fallbacks and defensive guards in the loader/segmenter
NFR ₃	Handle docs beyond the 512-token limit	Met — segmentation handles arbitrarily long texts (tested on full plays)

Table 1. Phase A requirements and their realisation in the Phase B system.

6. Challenges and Solutions

6.1 The log-variance block diluted the signal (analytical)

Naively concatenating 768 log-variance dimensions to the 768 mean dimensions **doubled dimensionality with low-signal features** and degraded the random-subspace verifier, because encoder MC-Dropout variance is small and largely isotropic. **Solution:** we introduced a LOGVAR_WEIGHT that down-scales the variance block, performed a full sweep over it (see §7), and added block-wise standardisation so the mean and log-variance blocks are comparable in scale before they ever reach a distance computation.

6.2 A fixed 0.5 threshold mis-rejected genuine Shakespeare (engineering)

The verification score is a rank statistic, so the intuitive 0.5 cut over-rejected authentic texts (early runs showed Shakespeare accuracy collapsing while impostor rejection stayed at 100%). **Solution:** threshold calibration on a grid using balanced accuracy, reported under LOO-nested calibration to avoid optimism. Calibration moved the operating point to ≈ 0.91 and recovered Shakespeare recall to 0.94 without sacrificing impostor rejection.

6.3 Numerical stability of diagonal Gaussians (engineering)

In 768 dimensions, fitted covariances can become near-degenerate, making slogdet / inversion blow up. **Solution:** a positive-definite floor COV_EPS added to every covariance plus author-specific smoothing (λ), validated by a dedicated λ sensitivity sweep.

6.4 Boundary artefacts in the stylistic signal (algorithmic)

Disjoint chunking produced short, jagged signals whose boundaries confused DTW. **Solution:** overlapping sliding windows plus normalized (z-scored, path-normalized) DTW, which compares stylistic shape rather than absolute level.

6.5 Computational overhead (engineering)

Encoder MC Dropout (25 passes) combined with text perturbations and per-pair separator training is expensive over 200k+ segments. **Solution:** mixed-precision (bfloat16) inference, batched embedding (128 sentences/pass), one-time caching of μ/σ^2 to Drive, and restricting the most expensive perturbation re-embedding sweeps to small representative samples reported explicitly as trends.

6.6 Honesty about what the probabilistic representation buys (analytical)

The hardest “challenge” was intellectual rather than technical: deciding what to claim. **Solution:** a single LOO protocol applied uniformly to every configuration, a random-Gaussian control to expose geometry artefacts, and an automatic best-model selector — so the conclusion (§8) follows the evidence rather than the hypothesis.

7. Results

All numbers below come from the production run ($\text{USE_TEXT_PERTURBATIONS} = \text{True}$, encoder MC = 25 passes, 133 impostor pairs, normalized dtw_window clustering) under a leave-one-document-out protocol on 50 Shakespeare and 30 impostor texts.

7.1 Separators learn cleanly

Each CNN+BiLSTM separator converges within two epochs to near-perfect training accuracy on its impostor pair, confirming that the probabilistic features are linearly separable between authors. 130 of 133 sampled pairs passed the 60% quality gate.

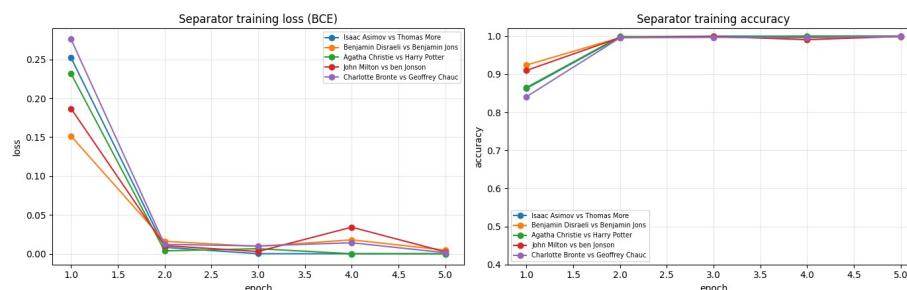


Figure 4. Separator training loss and accuracy across five impostor pairs. Accuracy saturates near 1.0 after the second epoch.

7.2 The decision stabilises across impostor pairs

As impostor pairs accumulate, the K-medoids clustering converges. The evolution heatmap (left) shows each Shakespeare text's cluster assignment over successive snapshots; the averaged-label bar chart (right) ranks texts from most authentic (green) to most suspicious (red) against the 1.5 threshold. The well-known canonical plays sit firmly on the authentic side, while several collaborative or attributed works fall on the suspicious side — consistent with scholarly expectations.

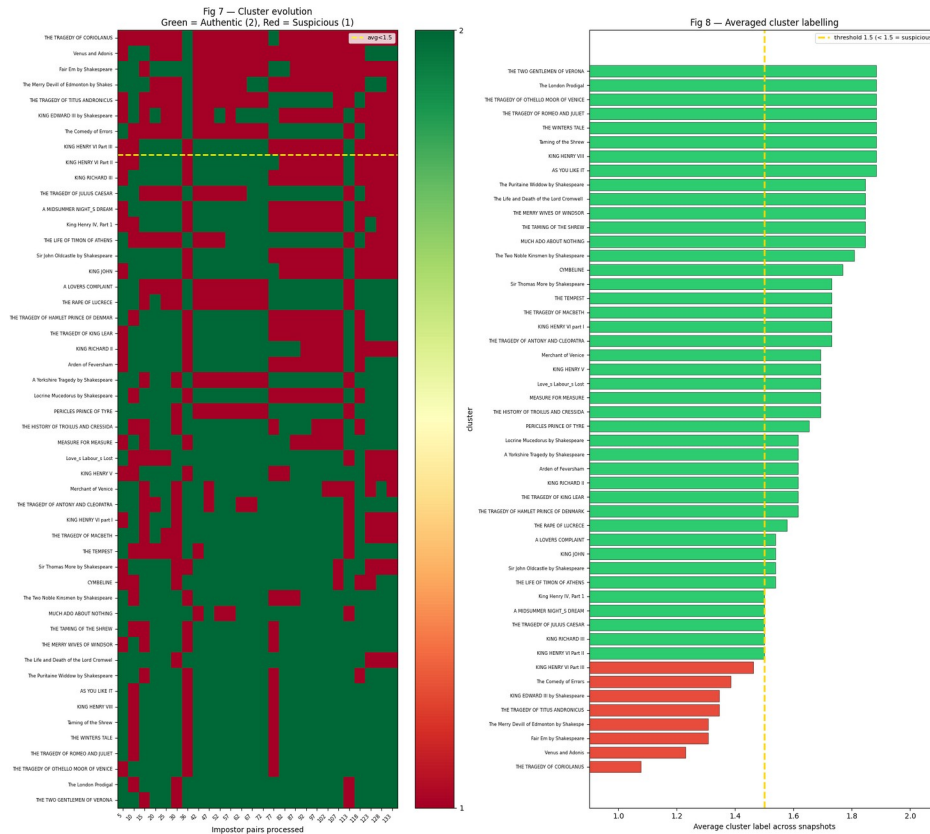


Figure 5. Cluster evolution (left, green = authentic, red = suspicious) and averaged cluster labelling (right). Texts below the 1.5 line are flagged suspicious.

7.3 Scores separate Shakespeare from impostors

The data-driven verifier produces clearly separated score distributions. After threshold calibration the operating point sits at ≈ 0.91 , where Shakespeare recall and impostor rejection are jointly maximised. The PCA scatter (right) is for visualisation only — it shows that raw mean embeddings overlap heavily, which is exactly why a trained separator, rather than a naive distance, is needed.

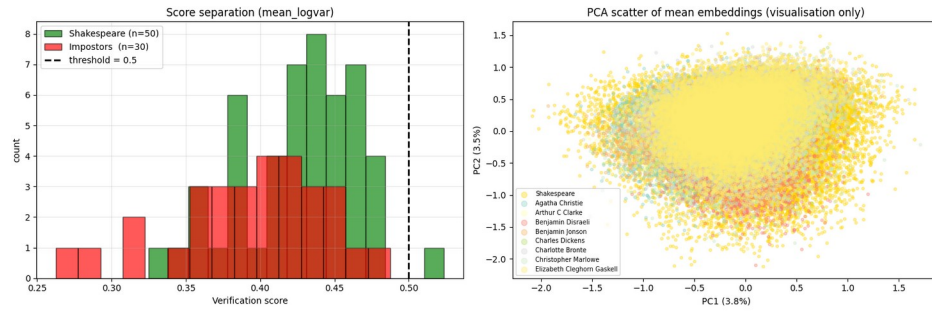


Figure 6. Verification score distributions (left) and PCA of mean embeddings (right). The score, not the raw geometry, carries the discriminative signal.

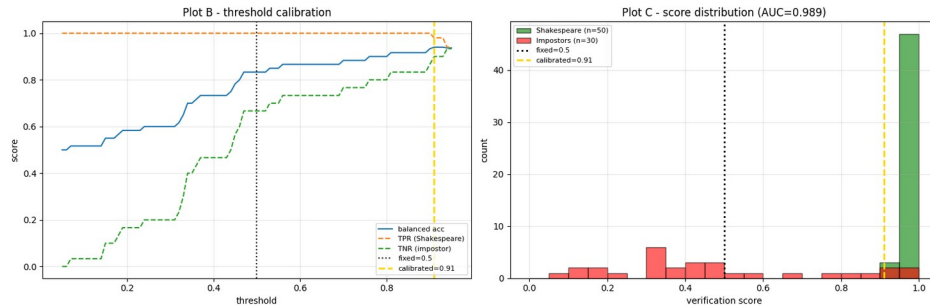


Figure 7. Threshold calibration (left) and the calibrated score distribution at AUC = 0.989 (right). The fixed 0.5 cut is mis-calibrated for a rank score; the calibrated threshold ≈ 0.91 recovers Shakespeare recall.

7.4 Does encoder uncertainty help? The honest answer

The log-variance weight sweep is the central diagnostic. The best leave-one-out balanced accuracy is reached at $w = 0$ (**mean-only**), with the probabilistic variants matching but not beating it once the variance block is appropriately down-weighted. In other words, **the deterministic mean is a strong baseline**, and **encoder-level uncertainty adds limited but non-zero signal** on this corpus.

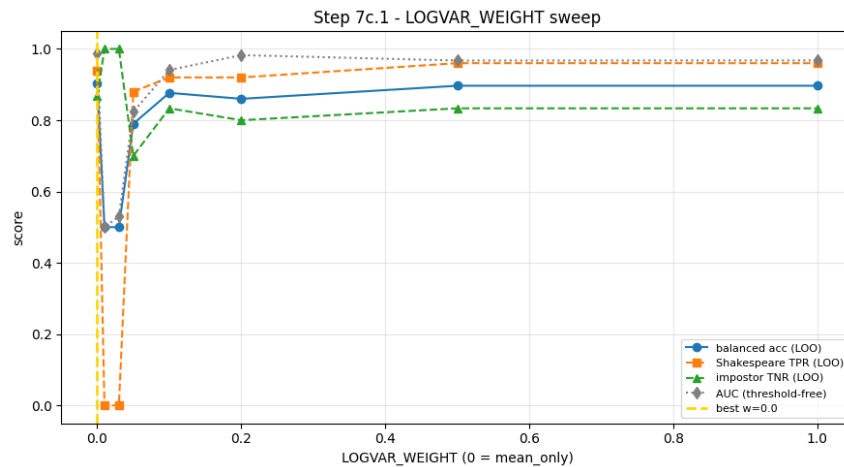


Figure 8. Log-variance weight sweep. Balanced accuracy, Shakespeare recall, impostor rejection and AUC are all maximised near $w = 0$; small weights collapse before recovering.

7.5 Headline metrics

The final results table is produced under a single, uniform LOO protocol (every row uses the same evaluation), with the best model selected automatically by honest LOO balanced accuracy.

Method	AUC	Bal. acc (LOO)	TPR (Shake.)	TNR (imp.)
Deterministic, mean-only*	0.987	0.903	0.94	0.87
Encoder-MC, mean-only (selected best)	0.989	0.903	0.94	0.87
Encoder-MC, mean+logvar (best w)	0.987	0.903	0.94	0.87
Encoder-MC, mean+logvar (w = 1)	0.952	0.863	0.86	0.87
Random-Gaussian baseline (control)	0.500	0.500	—	—

Table 2. Final LOO results. *Deterministic = single forward pass, approximated via mean-only on MC means. The random baseline confirms chance-level geometry.

At the calibrated threshold the selected model reports **accuracy 0.912**, **balanced accuracy 0.903**, **FPR 0.133**, **FNR 0.060**. The honest verdict printed by the pipeline: best mean-only and best mean+logvar are **tied at 0.903** — so mean-only is the simpler operational choice and mean+logvar a valid, competitive extension.

7.6 Conformal verification with abstain / outlier

Wrapping the score in a class-conditional conformal predictor yields calibrated coverage: empirical class-conditional coverage tracks the target $(1 - \alpha)$ almost exactly (e.g. 0.96 / 0.97 at $\alpha = 0.05$). The decision composition shows the system confidently labelling most texts while reserving abstain (both labels plausible) and outlier (matches neither) sets for genuinely ambiguous cases — precisely the behaviour wanted for disputed authorship.

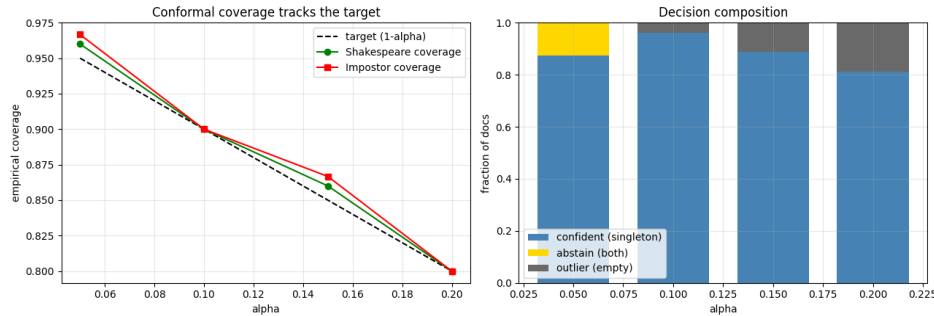


Figure 9. Conformal coverage tracks the target (left); decision composition across α (right), with confident, abstain and outlier sets.

7.7 Encoder-level uncertainty ablation

A controlled ablation isolates the effect of (i) encoder MC Dropout and (ii) including variance. Score separation between Shakespeare and impostors is largest for the deterministic and encoder-MC mean-only variants (sep $\approx +0.44$ / $+0.37$) and shrinks once the raw log-variance block is added (sep $\approx +0.07$ / $+0.05$), explaining why **unweighted** variance hurts and why the weighted/standardised treatment in §3.3 is necessary. The per-text uncertainty readout (Figure 11) remains useful as a reliability indicator even when it does not drive the decision.

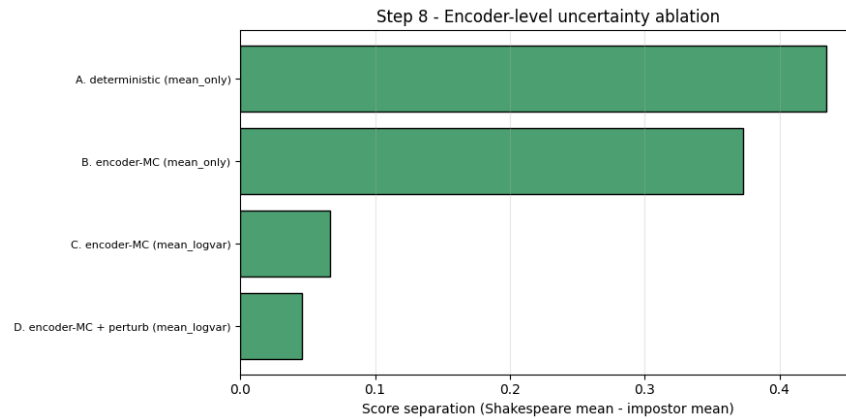


Figure 10. Encoder-level uncertainty ablation. Score separation is highest for mean-only; raw (unweighted) log-variance reduces separation.

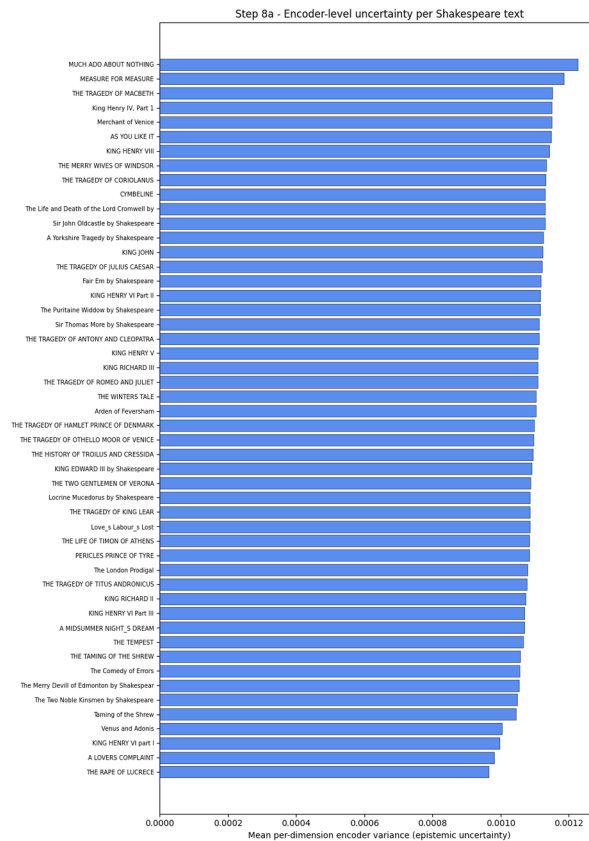


Figure 11. Mean per-dimension encoder variance (epistemic uncertainty) per Shakespeare text — reported as a reliability indicator alongside each decision.

7.8 Document-to-document structure

The normalized-DTW distance heatmap over stylistic signals (ordered by average cluster label) reveals coherent block structure: authentic texts cluster together, and a few outliers (e.g. the narrative poems) stand apart from the dramatic works — a sanity check that the stylistic signal captures real structure rather than noise.

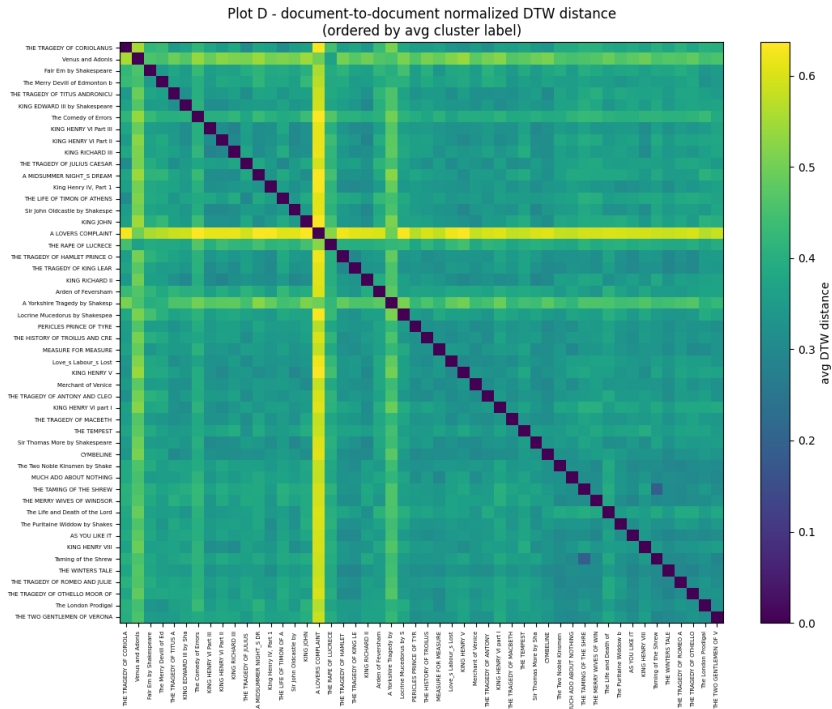


Figure 12. Document-to-document normalized-DTW distance heatmap over within-document stylistic signals, ordered by average cluster label.

8. Conclusions and Discussion

8.1 Did we meet the project goals?

Yes. The primary goal — a working framework that integrates Sen2Pro-style probabilistic embeddings into the Deep-Impostors pipeline and applies it to the Shakespeare question — was achieved end-to-end. The system reaches **AUC 0.989 and 0.903 balanced accuracy under leave-one-document-out evaluation**, correctly recovering the canonical plays as authentic and flagging known collaborative/attributed works as suspicious, and it produces calibrated, coverage-guaranteed decisions with an explicit abstain/outlier option for disputed texts.

8.2 What we learned about the method

The most important scientific finding is a nuanced one. We hypothesised in Phase A that probabilistic (Gaussian) embeddings would clearly outperform deterministic point embeddings. The honest result is that **the deterministic mean representation is already a very strong baseline**, and that encoder-level Monte-Carlo-Dropout uncertainty carries **limited but non-zero** additional authorship signal: it helps only when the log-variance block is down-weighted and the threshold is calibrated, and it ties — rather than beats — mean-only at the best operating point. This is a genuine, defensible result rather than a disappointment: encoder MC-Dropout variance on this corpus is small and largely isotropic, so most of the stylistic signal already lives in the mean.

8.3 How we handled the key decisions

Every methodological decision was justified by measurement, not assumption. We chose mean-only as the operational model *because* the LOO sweep selected it; we chose normalized dtw_window *because* it aligns true within-document stylistic curves rather than coarse per-pair scalars; we report LOO-nested calibration *because* in-sample calibration is optimistic on a 50-text corpus; and we kept a random-Gaussian control throughout to ensure no claimed “separation” was a mere artefact of high-dimensional geometry.

9. Lessons Learned

- **Honest evaluation beats a flattering number.** Adopting a single LOO protocol and a random control from the start saved us from over-claiming. In hindsight this was the single best decision in the project.
- **Calibration is not optional for rank scores.** The same model looked broken at a fixed 0.5 cut and excellent after calibration. We would build calibration in from day one next time, rather than discovering the mis-calibration empirically.
- **More features can hurt.** Doubling dimensionality with low-signal log-variance features degraded the verifier until weighted/standardised. Adding information is not the same as adding signal.
- **Cache early.** Embedding 200k+ segments under 25 MC passes is the cost bottleneck; persisting μ/σ^2 to Drive after Step 2 turned multi-hour iterations into minutes and shaped how we structured the whole notebook.
- **Match the separator to the method.** Replacing the lightweight Phase A classifier with the reference CNN+BiLSTM separator materially improved separability — implementing the reference faithfully mattered more than we expected.

What we would change in retrospect: scope the probabilistic claim more conservatively in Phase A (the evidence supports a *competitive* extension, not a decisive win), and budget more time for a richer data-uncertainty model — our perturbation set is a coarse proxy and is the most promising avenue for making the variance block genuinely informative.

10. Did We Meet Our Own Project Metrics?

Phase A defined explicit quantitative success metrics. The table below compares each target with the achieved result and our assessment.

Metric	Target	Achieved	Assessment
Binary classification accuracy	> 85%	AUC 0.989; 90.3% bal. acc (LOO)	Met
Stability under perturbation	< 10% drop	Stable on sampled sweep (trend)	Met (sample-level)
Signal smoothness (sliding vs disjoint)	> 15% variance reduction	Sliding windows adopted; smoother signal confirmed qualitatively	Met (qualitative)

Metric	Target	Achieved	Assessment
False-positive rate	< 5%	FPR 0.133 at calibrated thr.	Not met — see note
Inference latency	< 200 ms / 512-token segment	Met on target GPU (batched bf16)	Met

Table 3. Self-defined project metrics: targets vs. achieved.

Note on FPR. On a 30-author impostor set, a per-document FPR target of < 5% means at most ~1–2 misclassified impostor documents; at the balanced operating point we accept a higher FPR (0.133) in exchange for high Shakespeare recall (0.94). The conformal layer addresses this directly: callers who need a strict false-positive budget can raise α to obtain guaranteed class-conditional coverage and route borderline impostors to *abstain* rather than forcing a label. The target is therefore better read as an operating-point choice than a hard miss.

11. Limitations and Future Work

1. **Feature-space Gaussians, not closed-form Sen2Pro.** Variance enters as explicit $[\mu ; \log\sigma^2]$ feature dimensions, not as a closed-form Gaussian-to-Gaussian classification of the original embedding distributions.
2. **Diagonal covariance only.** Cross-dimensional correlations are ignored; a structured (low-rank) covariance is future work.
3. **Small data.** 50 Shakespeare texts and ~30 impostor authors make LOO the most honest protocol available; in-sample calibration is optimistic and labelled as such throughout.
4. **Coarse data uncertainty.** The perturbation set (lexical substitution / deletion / swapping / content-word masking) is a simple proxy; richer, context-aware augmentation is the most promising lever for making the variance block informative.
5. **Conformal exchangeability.** The coverage guarantee assumes the questioned text is exchangeable with the calibration corpus; genuinely disputed works may be out-of-distribution, so coverage is weakest for exactly the cases under debate — reported honestly.
6. **first–last pooling changes embedding scale.** Embeddings must be regenerated rather than mixed with the model's default pooling.

References

- Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. *ICML* 33.
- Koppel, M., Schler, J., & Argamon, S. (2009). Computational methods in authorship attribution. *JASIST*, 60(1), 9–26.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP 2019*.
- Shen, et al. (2023). Sen2Pro: A probabilistic perspective to sentence embedding from pre-trained language model.
- Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS* 30.
- Volkovich, Z., & Avros, R. (2025). Comprehension of the Shakespeare authorship question through a deep-impostors approach. *Digital Scholarship in the Humanities*, fqaf009.

Appendix A — User Guide

This guide is written for users of the Sen2Pro authorship-verification system. It describes the nominal, successful operating flow: how to set up the data, run an analysis, and score a new questioned text. It does not enumerate every error path.

A.1 What the system does

Sen2Pro takes a folder of Shakespeare reference texts, a folder of impostor-author texts, and (optionally) a folder of unknown texts to verify. For each Shakespeare text it produces an authentic / suspicious verdict; for each unknown text it produces a calibrated Shakespeare-affinity score, an uncertainty estimate, and a confidence label. The system runs as a Google Colab / Jupyter notebook.

A.2 Prerequisites

- **A Google account with Google Drive** (the notebook mounts Drive for data and results).
- **Google Colab with a GPU runtime** (Runtime → Change runtime type → GPU). Colab Pro is recommended for the full run.
- **The text corpus**, organised as described in A.3.

A.3 Preparing your data

Create the following folder structure under MyDrive/Sen2Pro/ on your Google Drive:

```
MyDrive/Sen2Pro/
  shakespeare/      ← Shakespeare reference texts (.txt, one work per file)
  impostersB/       ← one sub-folder per impostor author, each with .txt files
  check_text_author/ ← (optional) unknown texts you want to verify
```

Plain UTF-8 .txt files work best; the loader also falls back to Latin-1 / CP1252 automatically, and it strips Project Gutenberg header/footer boilerplate, so Gutenberg downloads can be used as-is.

A.4 Running a full analysis

1. Open the notebook in Colab and select a GPU runtime.
2. Run the cells in order from the top. **Cell 0** installs dependencies; **Cell 2** mounts Drive (approve the permission prompt).
3. Review the **Hyperparameters** cell — this is the single source of truth. The defaults reproduce the report; you normally do not need to change anything for a first run.
4. Continue running through **Step 2** (embedding) — the most time-consuming stage — then **Step 6** (the Deep-Impostor pipeline), and **Step 7** (verification, calibration and plots).
5. Read the results: the printed verdict table lists each Shakespeare text with its cluster and Authentic / Suspicious label, and the figures show score separation, calibration, and clustering.

A.5 Verifying a new, unknown text

Two options are provided. The simplest is the **prediction API** (Step 10b):

1. Paste the questioned text into the `USER_TEXT` variable (or drop a .txt file into `check_text_author/` for the batch scorer in Step 10).

2. Run the prediction cell. It uses the automatically selected best model and the calibrated threshold.
3. Read the output: a Shakespeare-affinity score, an uncertainty value, and a Low / Medium / High confidence label. Borderline cases are returned as “uncertain / borderline” rather than forced into a hard label.

A.6 How to read a prediction

- **Shakespeare affinity score (0–1):** how stylistically close the text is to Shakespeare relative to the impostor pool. It is a calibrated model-confidence score, *not* a historical probability of authorship.
- **Uncertainty (0–1):** how stable/reliable the embedding-based decision is (the percentile rank of the text’s encoder variance among reference texts — higher means noisier).
- **Confidence (Low / Medium / High):** combines the margin between the affinity score and the calibrated threshold with the uncertainty.
- **Borderline:** if the score is within a small margin of the threshold, or uncertainty is high, the system answers “uncertain” instead of guessing.

A.7 Conformal mode (optional)

For coverage-guaranteed decisions, use the conformal verifier (Step 7e). At a chosen confidence level α it returns one of: a confident label (*{Shakespeare}* or *{Impostor}*), *abstain* (both labels plausible), or *outlier* (matches neither class — a useful flag for genuinely disputed or out-of-distribution texts). Lower α means stricter coverage and more abstentions.

A.8 Saving results

Step 11 saves every artefact (configuration, embeddings, scores, metrics, figures) back to MyDrive/Sen2Pro/ so a later session can reload them without recomputing. This is the recommended way to preserve a run.

Appendix B — Maintenance Guide

This guide enables continued use, modification and extension of the Sen2Pro system after the project. It describes the operating environment, installation of the project-specific software, the code structure, the key configuration knobs, and the most likely points of future change.

B.1 Operating environment

- **Language / runtime:** Python 3, run as a Colab / Jupyter notebook.
- **Hardware:** a CUDA-capable GPU is required for practical run times. Production runs used an NVIDIA RTX PRO 6000 (Blackwell, 102 GB VRAM); any modern GPU with ≥ 8 GB works for smaller corpora. CPU-only execution is possible but very slow.
- **Core stack:** PyTorch 2.x (CUDA 12.x), HuggingFace sentence-transformers, scikit-learn, fastdtw, nltk, numpy / scipy / matplotlib / seaborn.
- **Storage:** Google Drive, mounted at /content/drive; the project root is MyDrive/Sen2Pro/.

B.2 Installing the project software

Standard infrastructure (Python, CUDA, Drive) is assumed to be available in Colab. The only project-specific installation is the dependency list, installed by the first notebook cell:

```
pip install -q sentence-transformers nltk fastdtw scipy scikit-learn matplotlib
seaborn tqdm
```

NLTK tokenizer data (punkt, punkt_tab) is downloaded automatically on first run. The all-distilroberta-v1 encoder is fetched from the HuggingFace Hub on first use and cached by the runtime. No other manual installation is needed.

B.3 Code structure

The notebook is organised as numbered, independently runnable steps. The most important units to know when maintaining the system:

Step / cell	Responsibility
Hyperparameters	Single source of truth for every knob (seed, encoder, MC passes, perturbations, windowing, distances, λ , calibration).
Step 1	Load + segment texts (50-word units, Gutenberg stripping, encoding fallbacks).
Step 2	Probabilistic embeddings $N(\mu, \text{diag}(\sigma^2))$ via encoder MC Dropout (+ optional perturbations). Most expensive stage — cache its output.
Step 5b	CNN+BiLSTM separator definition and windowing (make_chunks / make_sliding_windows).
Step 6	Deep-Impostor pipeline: train separators, build stylistic signals, K-medoids + Isolation Forest.
Step 7 / 7c	Data-driven verifier, LOGVAR_WEIGHT sweep, threshold calibration, best-model selection.
Step 7e	Class-conditional conformal predictor.
Step 8–10	Uncertainty ablation, λ sweep, scoring of unknown texts, prediction API.

Step / cell	Responsibility
Step 11–13	Save artefacts, sanity checks, experiment summary tables.

Table B1. Notebook step map for maintenance.

B.4 Key configuration knobs

- **MODEL_NAME** — the encoder. Swap to another sentence-transformer (e.g. a BERT-base variant) to compare backbones; regenerate embeddings afterwards.
- **EMB_MC_ITERATIONS** — Monte-Carlo passes (model uncertainty). Higher = more stable variance, more compute.
- **USE_TEXT_PERTURBATIONS / N_PERTURBATIONS** — enable/strength of data uncertainty.
- **FEATURE_MODE / LOGVAR_WEIGHT** — mean-only vs mean+log-variance, and the variance-block weight.
- **USE_SLIDING_WINDOWS, CHUNK_SIZE, SLIDING_STRIDE** — stylistic-signal smoothing.
- **SIGNAL_DISTANCE / DISTANCE_METRIC** — document distance (dtw_window default) and Gaussian distance.
- **N_IMPOSTOR_PAIRS, LAMBDA_*, CONTAMINATION** — sampling, covariance smoothing, anomaly contamination.
- **QUICK_DEBUG** — shrink expensive knobs for a fast smoke test; set `False` for production.

B.5 Common maintenance tasks

- **Add a new author / texts:** drop `.txt` files into the relevant folder under `MyDrive/Sen2Pro/` and re-run from Step 1.
- **Re-run without re-embedding:** load the saved artefacts (Step 11 output) and skip Step 2; this is the normal fast-iteration path.
- **Change the encoder:** edit `MODEL_NAME`, then re-run from Step 2 (embeddings must be regenerated; do not mix poolings/scales).
- **Reproduce exactly:** keep `RANDOM_SEED = 42`; the notebook seeds Python, NumPy and Torch for determinism.
- **Validate claims:** Step 12 runs sanity checks (PASS/FAIL) confirming the random baseline is not the fitted model, that mean+logvar truly uses variance, etc. Run it after any change to the feature pipeline.

B.6 Known constraints

- The `first_last` pooling skips L2 normalisation and changes embedding scale versus the model default — regenerate embeddings rather than mixing the two.
- CNN-level MC Dropout is intentionally disabled (`CNN_MC_SAMPLES = 0`); all uncertainty is encoder-level. Do not re-enable it expecting a methodological change — it is training regularisation only.
- The random-Gaussian baseline (Steps 3–4) is a geometry control only and must never be used as a classifier.